

Brac University
Department of Electrical & Electronic Engineering
Fall2025



Course Number : EEE282

Section : 01

Group No : 04

Lab Report

Experiment no: 08

Name of the experiment:

Solution of Non-Linear Equations

Name: Souvik Barman Ratul ID: 24121205

All Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24121205	Souvik Barman Ratul
2.	24121308	Alif Tamjid
3.	23221082	Sabrina Maliha
4.	24121019	Monazzi Siddique

Objective:

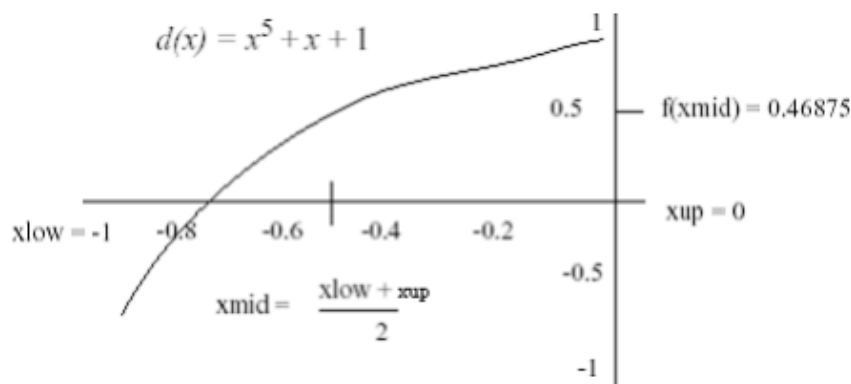
The main objective of this experiment is to determine the real roots of given nonlinear equations using numerical methods. In this experiment, the Bisection Method and the Newton–Raphson Method are applied to find approximate roots with a specified accuracy. Another objective is to understand how iteration, tolerance, and error control affect the convergence of numerical solutions. This experiment also aims to develop practical skills in implementing root-finding algorithms using MATLAB.

Theoretical Background:

Many non-linear equations that cannot be solved analytically. Numerical techniques are therefore used to obtain approximate solutions. In this experiment, different numerical methods for solving non-linear equations are studied, including the Bisection Method, False Position Method and Newton–Raphson Method.

Bisection Method

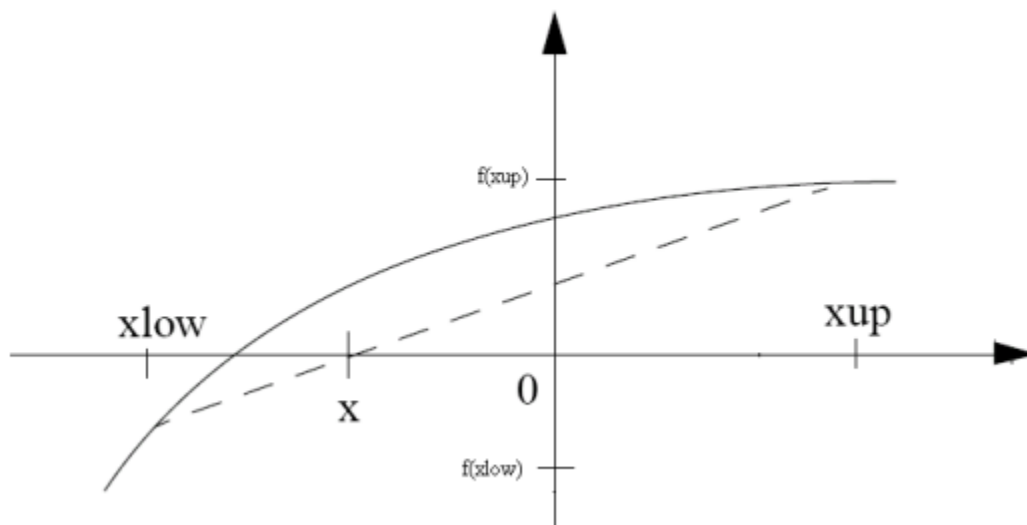
The Bisection Method is one of the simplest numerical procedures for finding the root of a function within a given interval. If a continuous function changes sign over an interval $[x_{\text{low}}, x_{\text{up}}]$ then at least one real root exists in that interval. The method determines the root by repeatedly halving the interval.



Based on the sign of the function at x_{mid} , the interval is updated and finds the small interval and gets the exact root value. The Bisection Method always converges if the initial interval is chosen correctly, but it usually takes many iterations because the interval is reduced slowly.

False Position Method

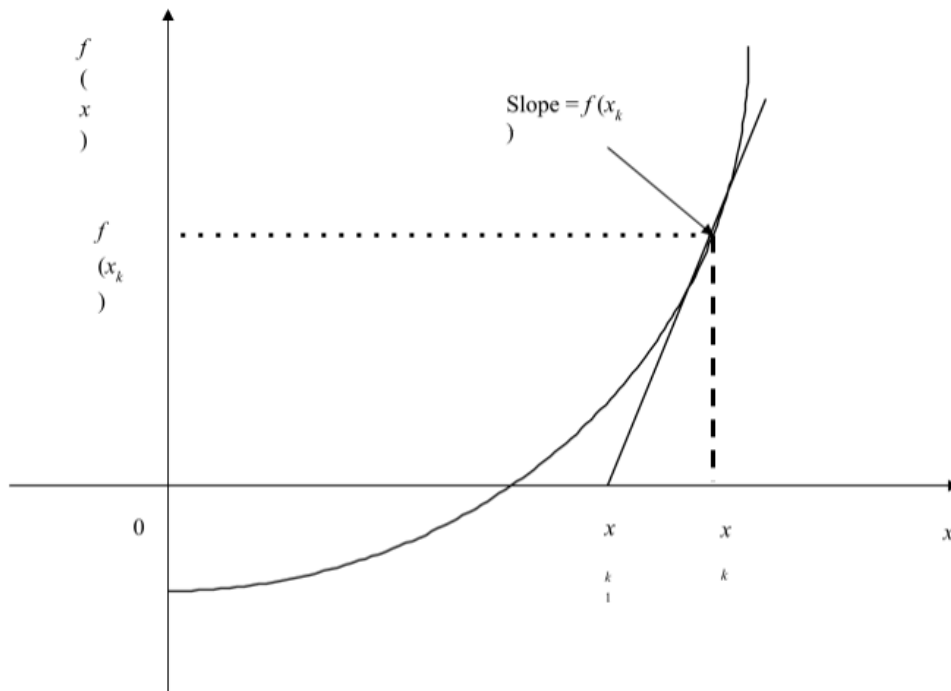
The False Position Method is similar to the Bisection Method, but instead of using the midpoint of the interval, it uses linear interpolation to estimate the root. A straight line is drawn between the function values at the two endpoints, and the point where the line crosses the x-axis is taken as the new approximation. This method often converges faster than the Bisection Method because it considers the function values, but in some cases convergence can still be slow.



$$x = x_{up} - \frac{f(x_{up})(x_{low} - x_{up})}{f(x_{low}) - f(x_{up})}$$

Newton–Raphson Method

The Newton–Raphson Method is a faster numerical method that uses the derivative of the function. Unlike the Bisection and False Position methods, it does not require the root to be bracketed. Starting from an initial guess, the method draws a tangent line at that point and uses the point where the tangent crosses the x-axis as a better estimate of the root.



The formula used is:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

If the initial guess is close to the actual root and the derivative is well-defined, the method converges very quickly.

Problem Statement:

Exercise-1: Find the real root of the equation $d(x) = x^5 + x + 1$ using the Bisection Method. $X_{\text{low}} = -1$, $X_{\text{up}} = 0$ and $\varepsilon = \text{selected } x \text{ tolerance} = 10^{-4}$.

Answer:

Code:

```
clear all;
close all;
clc;

f = @(x) (x.^5 + x + 1);

%%Bisection Method

%interval
xlow= -1;
xhigh=0;

e = 100;
tol=1e-4;

%variable for iteration
xr_prev=0;
iter=0;

if (f(xlow)*f(xhigh))>0
    disp(' No root in the interval');
else
    while(e>tol)
        iter=iter+1;

        xr= (xhigh+xlow)/2; % Midpoint

        % update interval base on sign
```

```

    if (f(xr)*f(xlow))<0
        xhigh=xr;
    else
        xlow=xr;
    end

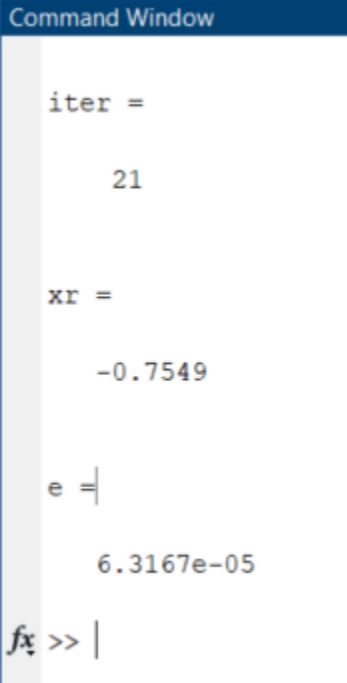
    %Compute approximate error
    e= abs((xr-xr_prev)/xr)*100;
    xr_prev=xr;

end
end

%final results
iter
xr
e

```

Output:-



Command Window

```

iter =

    21

xr =

   -0.7549

e =

   6.3167e-05

fx >> |

```

Explanation:

This code is written to find the root of the equation $f(x)=x^5+x+1$ using the Bisection Method, which is a simple and reliable numerical technique. Firstly, all previous variables, figures, and command window outputs are cleared by `clear all, close all, clc` to avoid any confusion. The function is then defined, and an initial interval $[-1,0]$ is selected where the root is expected to exist. Before starting the iteration the code checks the sign of the function at the lower and upper limits if there is no sign change, it concludes that no root lies in the chosen interval. When a valid interval is found, the program repeatedly calculates the midpoint of the interval and treats it as an approximate root. Based on the sign of the function at this midpoint, the interval is updated so that the root always remains inside it. After each iteration, the approximate relative error is calculated and compared with a predefined tolerance value. The iteration continues until the error becomes smaller than the tolerance, ensuring sufficient accuracy. Finally, the code displays the total number of iterations performed, the estimated root value, and the final error obtained.

Exercise-2: Use the Newton Raphson method to estimate the root of $f(x) = e^{-x} - 1$, employing an initial guess of $x = 0$. The tolerance is $= 10^{-8}$.

Answer:

Code:

```
clear all;
close all;
clc;

f = @(x) (exp(-x) - 1);
%%Newton_Raphson Method

h = 1e-6;
xr = 0;
tol = 1e-8;
e = 100;

% Iteration variables
iter = 0;

while (e > tol)
    iter = iter + 1;
    xr_prev = xr; % Store previous value

    %Approximate derivative by using FDD
    derivative = (f(xr_prev + h) - f(xr_prev)) / h;

    %Newton_Raphson update
    xr = xr_prev - f(xr_prev) / derivative;

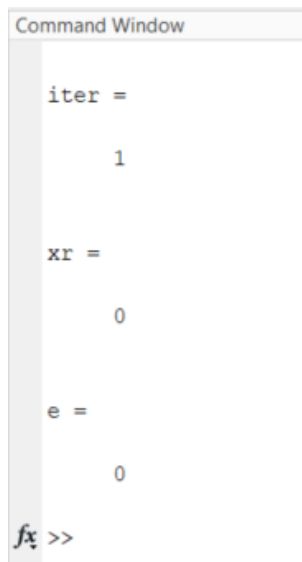
    e = abs(xr - xr_prev); % absolute error

end
```



```
% Final results
iter
xr
e
```

Output:-

A screenshot of the MATLAB Command Window. The window title is "Command Window". It displays the following output:

```
iter =  
      1  
  
xr =  
      0  
  
e =  
      0  
  
fx >>
```

Explanation:

Firstly, all previous variables, figures, and command window outputs are cleared by `clear all, close all, clc` to avoid any confusion. This code is used to find the root of the equation $f(x)=e^{-x}-1$ using the Newton–Raphson Method. A very small step size h is chosen to approximate the derivative using the forward difference method (FDD). An initial guess for the root is taken as $x_r=0$ and a tolerance value is set to control the required accuracy. The program then enters a loop where in each iteration the previous root value is stored and the derivative of the function is numerically approximated. Using this derivative, the Newton Raphson formula is applied to calculate a new estimate of the root. After each update, the absolute error between the current and previous estimates is computed. The loop continues until the error becomes smaller than the given tolerance. Finally, the code displays the total number of iterations, calculated root and the final error.

Discussion:-

In this experiment, different numerical methods were used to solve non-linear equations. The Bisection Method was very reliable because it always kept the root inside the interval, but it took more iterations to reach the answer. The False Position Method was slightly faster since it used the function values to estimate the root. The Newton Raphson Method gave the fastest result because it uses the slope of the function to quickly move toward the root. Overall, the Newton Raphson Method is better in terms of speed.